

Da file C ad Eseguibile

Riprendiamo ed approfondiamo alcuni concetti:

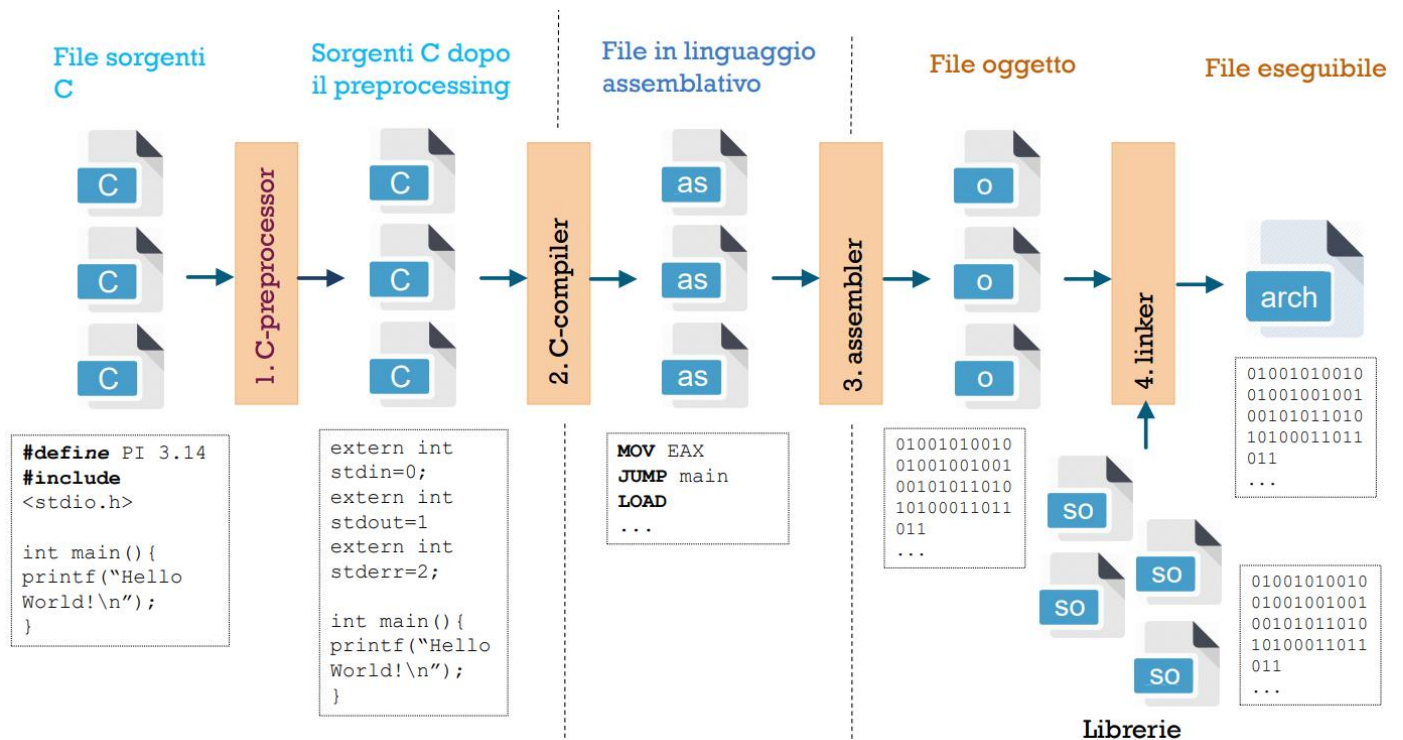
Per semplicità noi diciamo che “la macchina C interpreta ed esegue programmi scritti in linguaggio C”
In realtà però, **L'esecuzione dei programmi C è ottenuta attraverso un processo di Compilazione:**

- **Il C** è un linguaggio **non comprensibile all'architettura hardware** del processore: questa è in grado di eseguire solo programmi informatici scritti in linguaggio macchina;
 - Un programma in C **dovrà essere quindi Tradotto** in un programma equivalente **in un Linguaggio Macchina specifico di una architettura** target di processore (esempi: Intel x86, ARM, etc.), solo quest'ultimo potrà essere eseguito.
 - **Il processo di Traduzione si chiama Compilazione.**



In realtà il processo di **Compilazione** si compone di diversi passaggi:

1. Pre-processing dei File C (**preprocessore-C**);
2. Compilazione dei File Sorgenti C pre-processati, in linguaggio assembly (**compilatore-C**);
3. Traduzione dal Linguaggio Assembly al Linguaggio macchina (**assembler**), i file prodotti contengono **codice oggetto**;
4. Linking dei File Oggetto (**linker**).



Fase 1 - Pre-processing: Sostituzione

1. Riceve in input un programma scritto in C (o altri linguaggi)
2. Restituisce in output un altro file di testo dove:
 - a. ogni **Direttiva** presente è stata interpretata e rimpiazzata/espansa con il testo corrispondente

-E ferma l'esecuzione subito dopo il pre-processing.

```
gcc -E filename.c  
(per vedere l'output del preprocessore)
```

Direttive: #define, #include, #if, #ifdef, #ifndef

• Direttiva #include

- #include <nomefile.h>
 - Inserisce nel punto in cui si trova la direttiva il contenuto del file nomefile.h
 - La ricerca avviene solo nelle cartelle di sistema (es: /usr/include), e non nella directory corrente

- #include "filename.h"
 - La ricerca del file avviene prima nella directory corrente e poi nelle cartelle di sistema

```
gcc -I /home/schi/mylibs  
(per aggiungere una cartella di ricerca)
```

• Direttiva #define

- **Permette di definire** un nome simbolico per la definizione di:
 - una **Costante**,
 - una **Macro**,
 - una **Macro Parametrica**
 - Per convenzione, i nomi simbolici si definiscono con le lettere Maiuscole dell'alfabeto
 - Esistono alcune macro predefinite: `"__TIME__"`, `"__DATE__"`, `"__FILE__"`, `"__LINE__"`

Le Direttive solitamente sono composte da una sola linea; tuttavia è possibile definirne anche con più linee:

```
#define EXCHANGE(type,a,b) {\n    type aux ;\n    aux = a; \ a = b; \n    b = aux; }
```

Una Macro può essere anche definita a runtime quando si invoca il compilatore gcc:

```
gcc -D PI=3.14  
(equivalente a #define PI 3.14 nel file)
```

• Direttiva #if, #ifdef, #ifndef

- **E' possibile includere porzioni di codice in base:**
 - **alla valutazione di una espressione** che contiene costanti definite prima:

```
#if CONST > 10\n/* codice incluso se vero*/\n#else\n/* codice incluso se falso*/\n#endif
```

- **alla definizione (o meno) di una macro:**

```
#ifdef PLUTO\n/* codice inserito se\nPIPP0 è definito*/\n#endif
```

```
#ifndef PLUTO\n/* codice inserito se\nPIPP0 non è definito*/\n#endif
```

Fase 2 - Compilazione: Traduzione

Traduzione dei file sorgenti C pre-processati in Linguaggio Assemblativo (.s).

`-S` ferma l'esecuzione subito dopo la compilazione. `gcc -S filename.c`

□ ci sono moltissime opzioni:

- `-g` aggiunge le info di debug (usato per `gdb`)
- `-O2 -Os -O0` per i vari livelli di ottimizzazione
- `-std=c89` seleziona lo standard ANSI C (1989)
 - Variabili dichiarate solo all'inizio di un blocco (ad esempio, `for (int i=0; i<10; i++)`, non rispetta lo standard
 - I commenti inline non sono ammessi (solo `/* */`)
- `-Wall` visualizza tutti i warnings
- `-pedantic` non compila programmi non conformi con lo standard ANSI C
- `-m32 -marm` per compilare su altre architetture
- `-lm` per utilizzare le librerie matematiche

`gcc -S -g -O0 filename.c`

Fase 3 - Assemblaggio: Traduzione

Traduzione dal Linguaggio Assemblativo al Linguaggio Macchina

1. Riceve in input il file in Linguaggio Assemblativo
2. Restituisce in output un File Binario Oggetto

`-c` ferma l'esecuzione subito dopo l'assemblatore. `gcc -c filename.c`

`hexdump -C` per vedere il contenuto del file oggetto `hexdump -C filename.o`

Fase 4 - Linking: Eseguibile

1. Riceve in input uno o più file oggetto
 - a. **ATTENZIONE:** Un solo file in input deve avere definita la funzione `main()`

2. Restituisce un file eseguibile chiamato `a.out` `gcc filename.o filename2.o -o eseguibile`

- a. con l'opzione `-o` il nome può essere modificato